

Technical Documentation

Date: July 22, 2026, **Version:** 1.0

Revision History

Version	Date	Author	Description
1.0	22-july-2026	Prasanna B	Initial draft

Table of Contents

Technical Documentation	1
1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Intended Audience	3
1.4 References.....	4
1.5 Acronyms and Definitions.....	4
2. System Overview	5
2.1 High-Level Architecture	6
2.2 System flow - Key Components	6
2.3 User Roles and Access Levels.....	2
3. Functional Requirements.....	3
3.1 Ticket Management	3
3.2 Inputs/Outputs	3
3.3 Business Rules.....	6
3.4 Error Handling.....	7
4. Non-Functional Requirements.....	7
4.1 Performance	9
4.2 Security.....	10
4.3 Availability.....	11
4.4 Maintainability	11
5. Data Model / Database Design.....	12
5.1 Entity-Relationship Diagram	12
5.2 Tables and Fields.....	12
5.3 Indexes and Keys.....	13
5.4 Sample Data	14
6. Integration Points.....	14
6.1 External Systems	14
6.2 APIs Used	15
6.3 Data Flow Diagram.....	16
6.4 Authentication / Authorization.....	16

7. Test Strategy.....	17
7.1 Test Scenarios.....	17
7.2 Expected Results.....	18
7.3 Traceability to Requirements.....	18
8. Deployment & Configuration.....	19
8.1 Environments.....	19
8.2 Setup Steps	20
8.3 Configuration Files and Parameters.....	22
9. Appendix	23
Glossary.....	23
Screenshots.....	24
Code Snippets.....	26
References.....	28

1. Introduction

1.1 Purpose

This document provides the technical design and implementation details of the **AI Service Desk Backend Application**, developed using **FastAPI**. It serves as a comprehensive reference for developers, testers, reviewers, and future maintainers by documenting the application's architecture, functional modules, database design, API specifications, middleware, validation mechanisms, testing strategy, deployment process, and integration with external services.

The application enables users to manage support tickets through RESTful APIs while leveraging **Amazon Bedrock** to generate AI-powered ticket summaries and suggested responses. The backend follows a layered architecture that separates routing, business logic, persistence, and external service integrations to ensure maintainability and scalability.

This document also explains the implementation of:

- CRUD operations for ticket management
- Request and response validation using Pydantic
- Database persistence using PostgreSQL and SQLAlchemy ORM
- Database schema versioning using Alembic
- Middleware for response time measurement and CORS
- Global exception handling
- Unit testing using Pytest
- AI integration using Amazon Bedrock
- Local development using Docker
- Deployment and configuration guidelines

The purpose of this documentation is to provide sufficient technical knowledge for development, maintenance, testing, and future enhancements of the AI Service Desk backend.

1.2 Scope

This document covers the backend implementation of the **AI Service Desk Application** developed using **FastAPI**.

The current implementation provides a RESTful backend capable of managing support tickets while integrating AI services for automated ticket analysis.

The application currently supports the following features:

Ticket Management

- Create new support tickets
- Retrieve all available tickets
- Retrieve a specific ticket using its unique identifier
- Update ticket information
- Delete existing tickets

Data Validation

- Request validation using Pydantic models
- Custom validation rules
- Automatic response serialization
- UUID validation
- Enum validation for priority and status

Database Layer

- PostgreSQL database connectivity
- SQLAlchemy Async ORM integration
- Alembic database migrations
- Asynchronous database operations
- Automatic table creation during development

Middleware

- Response Time Middleware
- CORS Middleware
- Global Exception Handling
- Custom error responses

AI Integration

- Amazon Bedrock Runtime integration
- AI-generated ticket summaries
- Suggested customer responses
- Prompt template management
- Offline Fake Bedrock service for local development and testing

Testing

- Unit testing using Pytest
- Schema validation tests
- Service layer tests
- Mock-based AWS integration tests
- Happy path, edge case, and failure scenario testing

Deployment

- Docker containerization
- Environment-based configuration
- Local PostgreSQL setup
- Health and Readiness endpoints
- Swagger and ReDoc API documentation
- The scope of this document is limited to the backend application. Frontend implementation, authentication systems, authorization mechanisms, notification services, and production cloud deployment are outside the scope of the current implementation.

1.3 Intended Audience

This document is intended for the following stakeholders

Backend Developers

Developers responsible for implementing new features, maintaining existing APIs, fixing defects, and enhancing the application architecture.

QA Engineers

Engineers responsible for validating API functionality, preparing test cases, verifying business rules, and ensuring application quality.

Technical Reviewers

Architects and senior engineers responsible for reviewing the overall system design, implementation quality, coding standards, and architectural decisions.

Technical Mentors

Mentors evaluating the project implementation and providing technical guidance regarding backend architecture, software engineering best practices, testing methodologies, and cloud integrations.

DevOps Engineers

Engineers responsible for deployment, Docker configuration, environment management, database migrations, and application monitoring.

1.4 References

The following official references were used during the design and implementation of this project.

- [FastAPI Documentation](#)
- [SQLAlchemy Documentation](#)
- [PostgreSQL Documentation](#)
- [Alembic Documentation](#)
- [Pydantic Documentation](#)
- [Pytest Documentation](#)
- [Docker Documentation](#)
- [AWS Documentation](#)
- [Amazon Bedrock Documentation](#)
- [Boto3 Documentation](#)
- [Uvicorn Documentation](#)

These references provide implementation guidance, API specifications, and best practices followed throughout the development process

1.5 Acronyms and Definitions

Term	Description
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update and Delete operations
ORM	Object Relational Mapping
DTO	Data Transfer Object
UUID	Universally Unique Identifier
JSON	JavaScript Object Notation
SQL	Structured Query Language

Term	Description
CORS	Cross-Origin Resource Sharing
SQLAlchemy	Python ORM framework used for database interaction
Alembic	Database migration tool used with SQLAlchemy
Pydantic	Python library used for request validation and serialization
FastAPI	High-performance Python web framework
PostgreSQL	Open-source relational database management system
AWS	Amazon Web Services
Amazon Bedrock	Managed AWS service providing access to foundation AI models
Boto3	AWS SDK for Python
Middleware	Component executed before and after processing an HTTP request
Pytest	Python testing framework used for unit and integration testing
Docker	Platform for packaging and running applications inside containers

2. System Overview

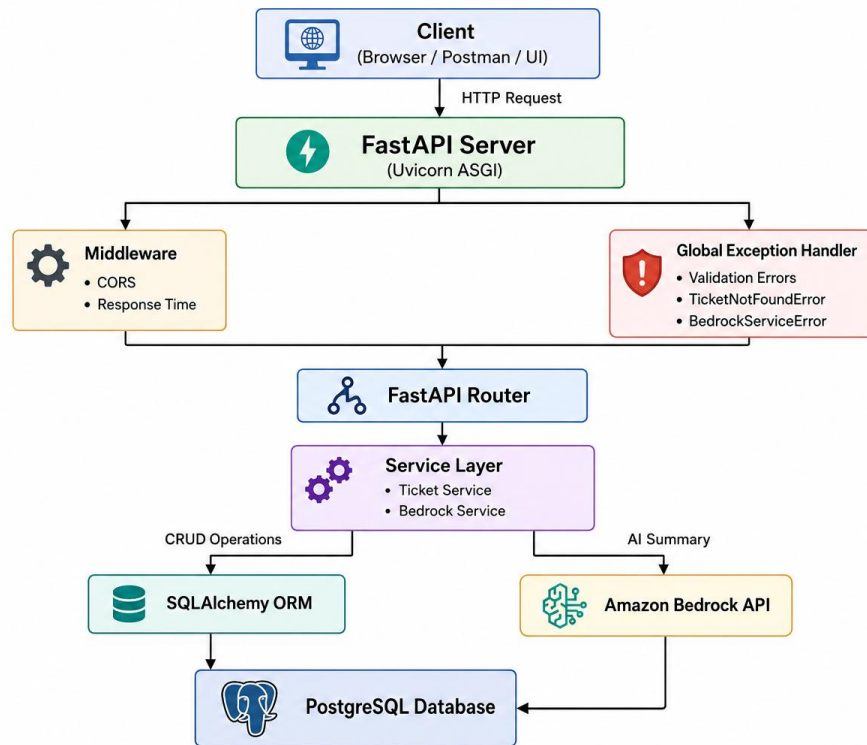
The AI Service Desk Backend is a RESTful web application developed using FastAPI. The application follows a layered architecture where each layer has a clearly defined responsibility.

Incoming client requests are first processed by middleware components responsible for handling cross-origin requests, measuring response time, and managing application-wide concerns. Requests are then routed to the appropriate API endpoints where request validation is performed using Pydantic models.

The validated requests are forwarded to the service layer, which contains the business logic responsible for ticket management and AI-based ticket summarization. Database operations are performed using SQLAlchemy Async ORM, while AI summarization requests are processed through Amazon Bedrock.

This separation of concerns improves readability, maintainability, scalability, and testability of the application

2.1 High-Level Architecture



2.2 System flow - Key Components

FastAPI Application

Acts as the main entry point of the application and initializes middleware, routes, exception handlers, and application lifecycle events.

Middleware

Processes every incoming request before it reaches the router. Middleware is responsible for measuring API response time, handling CORS, and attaching custom headers to outgoing responses.

Exception Handler

Captures validation errors, ticket-not-found exceptions, and unexpected server errors, converting them into standardized JSON responses.

Router Layer

Receives HTTP requests and forwards them to the corresponding service methods after validating request parameters.

Service Layer

Implements business rules, performs ticket management operations, communicates with the database, and invokes Amazon Bedrock for AI-based ticket summarization.

SQLAlchemy ORM

Maps Python models to PostgreSQL tables and performs asynchronous database operations including Create, Read, Update, and Delete (CRUD).

PostgreSQL Database

Stores all ticket-related information including ticket title, priority, status, assignee, and creation timestamp.

Amazon Bedrock

Processes ticket descriptions using foundation models to generate concise summaries and suggested customer responses.

2.3 User Roles and Access Levels

The current implementation does not include authentication or authorization mechanisms.

All API endpoints are publicly accessible for development and testing purposes.

Future enhancements may include:

- JWT-based authentication
- Role-Based Access Control (RBAC)
- Admin and Support Engineer roles
- Fine-grained API authorization
- Secure access to AI summarization endpoints

3. Functional Requirements

3.1 Ticket Management

Purpose

The Ticket Management module is the core functionality of the AI Service Desk Backend. It enables users to create, retrieve, update, and delete support tickets through RESTful APIs. The module follows the CRUD (Create, Read, Update, Delete) design pattern and provides a standardized interface for managing support requests.

The APIs are implemented using FastAPI and follow REST principles. Each request undergoes validation using Pydantic models before being processed by the service layer. Business logic is separated from the API layer to improve maintainability and scalability.

The module currently supports the following operations:

- Create a new ticket
- Retrieve all tickets
- Retrieve a specific ticket using its unique identifier
- Update an existing ticket
- Delete an existing ticket

These operations interact with the PostgreSQL database through SQLAlchemy's asynchronous ORM.

3.2 Inputs/Outputs

The AI Service Desk application provides RESTful APIs for managing support tickets and generating AI-powered ticket summaries. The following examples illustrate the request and response formats for the major API operations.

Create Ticket

Input

```
{  
  "title": "Server Issue",  
  "priority": "high",  
  "assignee": "prasanna"  
}
```

Output

```
{
```

```
"id": "7bb8df2b-2eb4-4af0-aaf5-ec6d2fa7e54",  
"title": "Server Issue",  
"priority": "high",  
"status": "open",  
"assignee": "prasanna",  
"created_at": "2026-07-23T10:45:33Z"  
}
```

Retrieve All Tickets

Request

GET /tickets

Response

```
[  
  {  
    "id": "...",  
    "title": "Server Issue",  
    "priority": "high",  
    "status": "open",  
    "assignee": "prasanna",  
    "created_at": "2026-07-23T10:45:33Z"  
  }  
]
```

Retrieve Ticket by ID

Request

GET /tickets/{ticket_id}

Response

```
{  
  "id": "...",  
  "title": "Server Issue",  
  "priority": "high",  
  "status": "open",  
}
```

```
"assignee": "prasanna",  
"created_at": "2026-07-23T10:45:33Z"  
}
```

Update Ticket

Input

```
{  
  "status": "closed",  
  "assignee": "john"  
}
```

Output

```
{  
  "id": "...",  
  "title": "Server Issue",  
  "priority": "high",  
  "status": "closed",  
  "assignee": "john",  
  "created_at": "2026-07-23T10:45:33Z"  
}
```

Delete Ticket

Request

DELETE /tickets/{ticket_id}

Response

```
{  
  "message": "Ticket deleted successfully"  
}
```

AI Ticket Summarization

Input

```
{  
  "ticket_description": "The printer in the finance department is showing a paper jam error."  
}
```

```
}
```

Output

```
{
```

```
  "summary": "The finance department printer is experiencing a persistent paper jam issue.",
```

```
  "suggested_response": "The issue has been received and our support team is investigating it. We will provide an update shortly."
```

```
}
```

3.3 Business Rules

The application enforces the following business rules to ensure data integrity, consistency, and reliable ticket management.

Ticket Creation

- The **title** field is mandatory.
- The title cannot be blank or contain only whitespace.
- The title must contain a minimum of **three characters**.
- Each ticket is automatically assigned a unique UUID.
- The ticket creation timestamp is generated automatically by the system.
- If no status is provided, the ticket is assigned the default status **Open**.
- The **assignee** field is optional during ticket creation.

Ticket Priority

The priority field accepts only the following values:

- Low
- Medium
- High

Any other value results in a validation error.

Ticket Retrieval

- Only valid ticket identifiers can be retrieved.
- Requests for non-existent tickets return a custom **Ticket Not Found** response.

Ticket Update

- Only existing tickets can be updated.
- Fields that are omitted in the request remain unchanged.
- Invalid request data is rejected during validation.

Ticket Deletion

- Only existing tickets can be deleted.
- Attempting to delete a non-existent ticket returns an appropriate error response.

AI Ticket Summarization

- Ticket descriptions must not be empty.
- Amazon Bedrock must return a valid JSON response.
- Invalid or malformed AI responses are converted into application-specific exceptions

3.4 Error Handling

4. Non-Functional Requirements

The application implements centralized exception handling to ensure consistent API responses and improve debugging.

Validation Errors

Incoming requests are validated using **Pydantic** models before reaching the business logic.

Validation failures include:

- Missing required fields
- Blank titles
- Invalid UUID values
- Invalid priority values
- Incorrect request formats

These errors are handled automatically by FastAPI and return:

HTTP Status Code: 422 – Unprocessable Entity

Example Response

```
{
  "detail": [
    {
      "loc": ["body", "title"],
      "msg": "Title must contain at least 3 characters."
    }
  ]
}
```

Ticket Not Found

When a requested ticket does not exist, the application raises a custom `TicketNotFoundError`.

HTTP Status Code: 404 – Not Found

Example Response

```
{
  "error": "ticket_not_found",
  "id": "<ticket_id>"
}
```

Bedrock Service Errors

The application communicates with Amazon Bedrock to generate AI-powered ticket summaries.

If the service encounters:

- Invalid AWS credentials
- Network connectivity failures
- Invalid model identifiers
- AWS ClientError

- AWS BotoCoreError

the application raises a custom BedrockServiceError.

HTTP Status Code: 500 – Internal Server Error

Example Response

```
{  
  "error": "Bedrock request failed"  
}
```

Invalid AI Response

If Amazon Bedrock returns malformed or unexpected JSON, the application raises a BedrockServiceError.

Example Response

```
{  
  "error": "Bedrock returned an invalid response"  
}
```

Unexpected Server Errors

Any unhandled runtime exceptions are captured by the global exception handler and returned as a standardized error response.

HTTP Status Code: 500 – Internal Server Error

Example Response

```
{  
  "error": "Internal Server Error"  
}
```

4.1 Performance

The application is designed to provide fast and efficient processing of API requests while maintaining scalability and responsiveness.

Performance-related features include:

- Asynchronous request handling using FastAPI.

- Asynchronous database operations using SQLAlchemy Async ORM.
- PostgreSQL database for efficient data storage and retrieval.
- Response Time Middleware that measures the execution time of every HTTP request.
- Lightweight REST APIs with JSON serialization.
- AI summarization performed only when explicitly requested.

Every API response includes an **X-Response-Time** header indicating the total time taken to process the request.

The application was also profiled during development using **PyInstrument** to analyze endpoint performance and identify potential bottlenecks.

4.2 Security

The current implementation includes several security best practices suitable for development environments.

Implemented security features include:

- Request validation using Pydantic models.
- Automatic input sanitization through schema validation.
- UUID-based resource identification.
- Environment variables used to store database credentials and AWS configuration.
- Global exception handling to prevent exposure of internal application details.
- CORS configuration to restrict cross-origin requests to trusted origins.

The following features are planned for future releases:

- JWT Authentication
- Role-Based Access Control (RBAC)
- Password encryption
- API authorization
- Rate limiting
- HTTPS enforcement

4.3 Availability

The application is deployed as a FastAPI service using the Uvicorn ASGI server.

Availability features include:

- Health endpoint (/health) for monitoring application status.
- Readiness endpoint (/ready) to verify database connectivity.
- Automatic database connection initialization during application startup.
- Docker support for consistent deployment across environments.

At present, the application does not implement:

- Load balancing
- High availability
- Failover mechanisms
- Multi-region deployment

These capabilities can be incorporated in future production deployments.

4.4 Maintainability

The application follows a modular layered architecture that separates responsibilities into independent components, improving readability, maintainability, and scalability.

The architecture consists of the following layers:

- **Router Layer** – Handles HTTP requests and responses.
- **Service Layer** – Contains business logic and AI integration.
- **Database Model Layer** – Defines SQLAlchemy ORM models.
- **Schema Layer** – Implements request and response validation using Pydantic.
- **Core Layer** – Manages configuration, middleware, exception handling, and database connectivity.
- **Test Layer** – Contains unit tests implemented using Pytest.

Additional maintainability features include:

- Database schema versioning using Alembic migrations.
- Dependency Injection for database sessions.
- Environment-based configuration using .env.
- Offline Fake Bedrock service for development and testing.
- Unit tests covering happy paths, edge cases, and failure scenarios.
- Docker support for reproducible development environments.
- Clear separation between business logic, persistence, and external service integrations.

This architecture enables new features to be added with minimal changes to existing modules while maintaining high code quality and testability.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

The AI Service Desk Backend currently uses a relational database implemented with **PostgreSQL**. The application maintains a single primary entity, **Ticket**, which stores all information related to service desk requests. SQLAlchemy ORM is used to map the Ticket entity to the corresponding database table.

At the current stage of development, there are no relationships with additional entities such as Users, Departments, or Categories. Therefore, the Entity-Relationship (ER) diagram consists of a single table.

5.2 Tables and Fields

Field	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier for each ticket
title	String	Not Null	Title of the support request
priority	String	Not Null	Priority level (Low, Medium, High)

Field	Data Type	Constraints	Description
status	String	Not Null	Current status of the ticket
assignee	String	Nullable	Assigned support engineer
created_at	Timestamp	Not Null	Ticket creation timestamp

5.3 Indexes and Keys

The current database design implements a simple indexing strategy suitable for the application's scope.

Primary Key

- **id** serves as the primary key of the Ticket table.
- Implemented as a UUID to ensure globally unique ticket identifiers.
- Automatically indexed by PostgreSQL for efficient record retrieval.

Foreign Keys

The current implementation does not contain foreign key relationships because only a single database entity has been implemented.

Additional Indexes

No additional indexes have been created in the current version of the application.

As the application scales, indexes may be introduced on frequently queried columns such as:

- status
- priority
- assignee
- created_at

to improve query performance.

5.4 Sample Data

Field	Value
id	7bb8df2b-2eb4-4af0-aaf5-ec6d2fa7e54
title	Email server is not responding
priority	High
status	Open
assignee	Prasanna
created_at	2026-07-23 10:45:33

6. Integration Points

6.1 External Systems

The AI Service Desk Backend integrates with several external systems and software components to provide data persistence, AI-powered functionality, and application hosting.

External System	Purpose
PostgreSQL	Stores ticket information persistently
Amazon Bedrock	Generates AI-powered ticket summaries and suggested responses
AWS Boto3 SDK	Enables communication with Amazon Bedrock Runtime
Docker	Provides containerized application deployment
Swagger UI	Interactive API documentation and testing
ReDoc	Alternative API documentation interface

During development and testing, a **Fake Bedrock Service** is used to simulate Amazon Bedrock responses without requiring AWS credentials or internet connectivity.

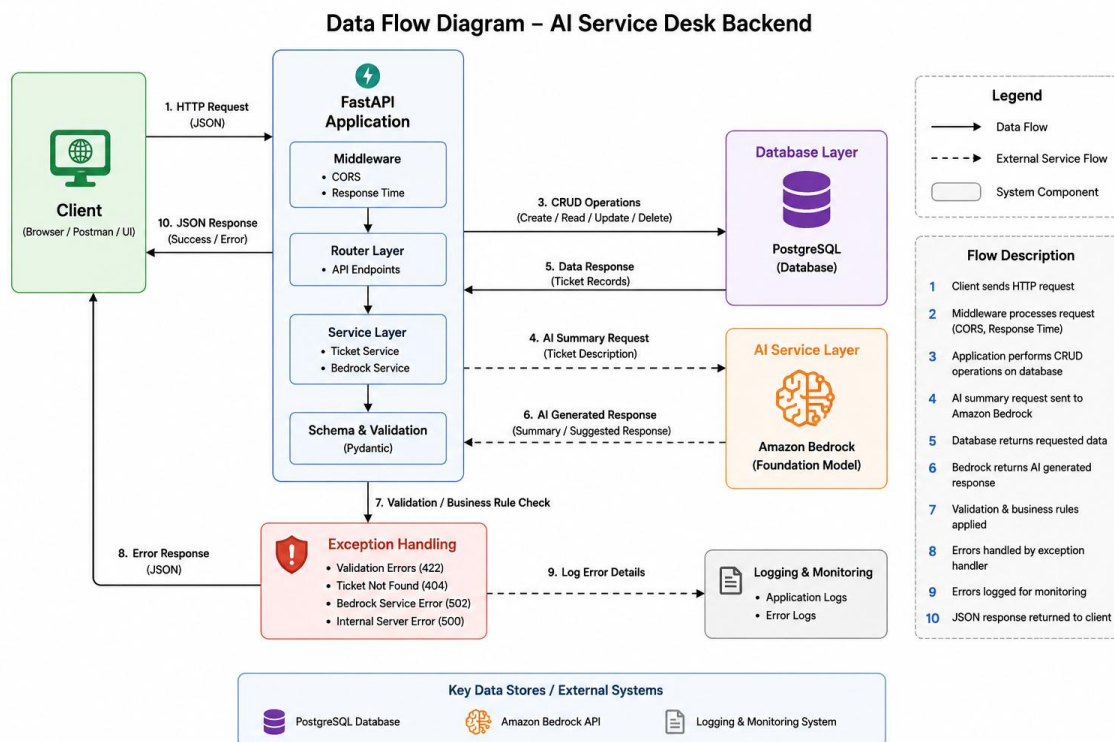
6.2 APIs Used

The application exposes RESTful APIs for ticket management and system monitoring.

Method	Endpoint	Purpose
POST	/tickets	Create a new ticket
GET	/tickets	Retrieve all tickets
GET	/tickets/{id}	Retrieve a ticket by its unique identifier
PATCH	/tickets/{id}	Update an existing ticket
DELETE	/tickets/{id}	Delete a ticket
GET	/health	Verify application health
GET	/ready	Verify database readiness
POST	/ai/summarize <i>(if implemented)</i>	Generate AI summary for a ticket description

The application also consumes the **Amazon Bedrock Converse API** to process ticket descriptions and return AI-generated summaries.

6.3 Data Flow Diagram



6.4 Authentication / Authorization

The current implementation does not include authentication or authorization mechanisms.

All API endpoints are publicly accessible and intended for development and testing purposes.

Future enhancements may include:

- JWT-based authentication
- OAuth 2.0 integration
- Role-Based Access Control (RBAC)
- User login and registration
- Refresh token mechanism
- API key authentication for external integrations

These features will improve application security and restrict access based on user roles and permissions.

7. Test Strategy

7.1 Test Scenarios

The application has been tested using **Pytest** to verify the correctness of business logic, API functionality, validation rules, and AI service integration.

Module	Test Scenario
Schema Validation	Blank title validation
Schema Validation	Minimum title length validation
Schema Validation	Valid request validation
Ticket Service	Create ticket
Ticket Service	Retrieve all tickets
Ticket Service	Retrieve ticket by ID
Ticket Service	Update ticket
Ticket Service	Delete ticket
Exception Handling	Ticket not found
Health API	Health endpoint verification
Readiness API	Database readiness verification
Bedrock Service	Successful AI summary generation
Bedrock Service	Invalid JSON response
Bedrock Service	AWS ClientError handling
Fake Bedrock	Local AI response simulation

7.2 Expected Results

Test Scenario	Expected Result
Blank title	HTTP 422 Validation Error
Title shorter than 3 characters	HTTP 422 Validation Error
Valid ticket creation	Ticket created successfully
Retrieve ticket	Ticket details returned
Retrieve invalid ticket	HTTP 404 Ticket Not Found
Update ticket	Updated ticket information returned
Delete ticket	Success confirmation message
Health endpoint	Status = OK
Readiness endpoint	Database connection verified
Bedrock summary	Summary and suggested response generated
Invalid AI response	BedrockServiceError returned
AWS communication failure	HTTP 500 Error Response

7.3 Traceability to Requirements

Functional Requirement	Test Coverage
Create Ticket	Ticket creation test
Retrieve Ticket	Get ticket by ID test
Retrieve All Tickets	Get all tickets test
Update Ticket	Update ticket test
Delete Ticket	Delete ticket test

Functional Requirement	Test Coverage
Request Validation	Schema validation tests
Exception Handling	Ticket Not Found test
Health Monitoring	Health API test
Database Readiness	Ready API test
AI Ticket Summarization	Bedrock service tests
Performance Middleware	Response Time Middleware verification

This traceability ensures that every implemented feature is verified through one or more automated test cases, improving confidence in the application's correctness and reliability.

8. Deployment & Configuration

8.1 Environments

The AI Service Desk Backend is designed to support deployment across multiple environments to facilitate development, testing, and production operations. Each environment serves a specific purpose during the software development lifecycle.

The **Development (Dev)** environment is primarily used by developers for implementing new features, debugging, and local testing. It is configured with development settings, local databases, and optional debugging tools to enable rapid application development.

The **Quality Assurance (QA)** environment is used by testers to validate application functionality before release. This environment closely resembles the production environment and is intended for executing functional, integration, and regression tests to ensure software quality.

The **User Acceptance Testing (UAT)** environment provides a platform where stakeholders or end users can verify that the application satisfies business requirements. It allows users to evaluate application functionality before deployment to production.

The **Production (Prod)** environment hosts the live application used by end users. This environment is optimized for stability, performance, and security. Production deployments typically include monitoring, logging, automated backups, and secure configuration management.

For the current implementation of the AI Service Desk Backend, development activities are performed using a local PostgreSQL database and FastAPI server. Docker support is also provided to simplify deployment and ensure consistency across different development environments.

8.2 Setup Steps

The following steps describe the process required to set up and execute the AI Service Desk Backend in a local development environment.

Prerequisites

Before running the application, ensure that the following software is installed on the system:

- Python 3.12 or later
- PostgreSQL
- Git
- UV (or pip)
- Docker (optional)

Step 1: Clone the Repository

Clone the project repository from the version control system and navigate to the project directory.

```
git clone <repository-url>
```

```
cd ai-service-desk
```

Step 2: Create a Virtual Environment

Create and activate a Python virtual environment to isolate project dependencies.

Windows

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

Linux/macOS

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

Step 3: Install Dependencies

Install all required project dependencies.

Using **UV**:

```
uv sync
```

Or using **pip**:

```
pip install -r requirements.txt
```

Step 4: Configure PostgreSQL

Create a PostgreSQL database for the application.

Example configuration:

Parameter	Value
Database Name	ai_service_desk
Username	postgres
Password	*****
Port	5432

Step 5: Configure Environment Variables

Create a .env file in the project root directory and configure the required environment variables.

Example:

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/ai_service_desk
```

```
AWS_ACCESS_KEY_ID=<your-access-key>
```

```
AWS_SECRET_ACCESS_KEY=<your-secret-key>
```

```
AWS_REGION_NAME=us-east-1
```

```
BEDROCK_MODEL_ID=<model-id>
```

```
AWS_DEMO_MODE=false
```

```
SECRET_KEY=<your-secret-key>
```

```
DEBUG=true
```

Step 6: Run Database Migrations

Execute Alembic migrations to create the required database tables.

alembic upgrade head

Step 7: Start the Application

Run the FastAPI application using the Uvicorn ASGI server.

```
uvicorn app.main:app --reload
```

The application will be accessible at:

Service	URL
Application	http://127.0.0.1:8000
Swagger UI	http://127.0.0.1:8000/docs
ReDoc	http://127.0.0.1:8000/redoc

Step 8: Docker Deployment (Optional)

The project also supports containerized deployment using Docker.

Build the Docker image:

```
docker build -t ai-service-desk .
```

Run the container:

```
docker run -p 8000:8000 ai-service-desk
```

Docker provides a consistent execution environment and simplifies application deployment across multiple systems.

8.3 Configuration Files and Parameters

The application uses environment variables to manage configuration settings. Sensitive information such as database credentials and AWS configuration is stored outside the source code to improve security and flexibility.

The primary configuration file is the .env file located in the project root directory.

The following configuration parameters are currently supported:

Parameter	Description
DATABASE_URL	PostgreSQL database connection string
AWS_ACCESS_KEY_ID	AWS access key for Amazon Bedrock

Parameter	Description
AWS_SECRET_ACCESS_KEY	AWS secret access key
AWS_REGION_NAME	AWS region used for Bedrock
BEDROCK_MODEL_ID	Foundation model identifier
AWS_DEMO_MODE	Enables Fake Bedrock Service for local testing
SECRET_KEY	Application secret key
DEBUG	Enables development mode

9. Appendix

Glossary

Term	Description
FastAPI	High-performance Python framework for building REST APIs
PostgreSQL	Open-source relational database management system
SQLAlchemy	Python ORM framework used for database operations
Alembic	Database migration tool used with SQLAlchemy
Pydantic	Data validation and serialization library
Middleware	Software component executed before and after processing an HTTP request
CRUD	Create, Read, Update, Delete operations
REST API	Web service following REST architectural principles
ORM	Object Relational Mapping
UUID	Universally Unique Identifier
Docker	Containerization platform used for application deployment
AWS	Amazon Web Services

Term	Description
Amazon Bedrock	AWS managed service providing access to foundation AI models
Boto3	AWS SDK for Python
Pytest	Python testing framework

Screenshots

Profiler

```
async def profile_requests(request: Request, call_next):
    profiler = Profiler()
    profiler.start()

    response = await call_next(request)

    profiler.stop()

    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write("\n" + "=" * 80 + "\n")
        f.write(f"{request.method} {request.url.path}\n")
        f.write("=" * 80 + "\n")
        f.write(profiler.output_text(unicode=True, color=False))
        f.write("\n")

    return response
```

You, 24 minutes ago • file structure were c

Pydantic Schema Validation

```
class CreateTicketRequest(BaseModel):
    title: str
    priority: Literal["low", "medium", "high"]
    assignee: str | None = None

    @field_validator("title")
    @classmethod
    def validate_title(cls, value: str):
        value = value.strip()

        if not value:
            raise ValueError("Title cannot be blank.")

        if len(value) < 3:
            raise ValueError("Title must be at least 3 characters.")

        return value
from typing import Optional, Literal
```

Locust Load Testing

```
from locust import HttpUser, between, task
You, 13 hours ago | 1 author (You)
class TicketApiUser(HttpUser):
    wait_time = between(1, 3)

    @task(3)
    def check_health(self) -> None:
        self.client.get("/health", name="GET /health")

    @task(2)
    def list_tickets(self) -> None:
        self.client.get("/tickets", name="GET /tickets")

    @task(1)
    def create_ticket(self) -> None:
        self.client.post(
            "/tickets",
            name="POST /tickets",
            json={
                "title": "Load test ticket",
                "Priority": "3",
                "assignee": "prasanna",
            },
        )
```

Code Snippets

Create Ticket

```
@router.post("", response_model=TicketResponse)
async def create_ticket(
    ticket: CreateTicketRequest,
    db: AsyncSession = Depends(get_db),
):
    return await ticket_service.create_ticket(ticket, db)
```

Get Ticket by Id

```
@router.get("/{ticket_id}", response_model=TicketResponse)
async def get_ticket(
    ticket_id: UUID,
    db: AsyncSession = Depends(get_db),
):
    ticket = await ticket_service.get_ticket(ticket_id, db)

    if ticket is None:
        raise TicketNotFoundError(str(ticket_id))

    return ticket
```

Delete Ticket by Id

```
@router.delete("/{ticket_id}")
async def delete_ticket(
    ticket_id: UUID,
    db: AsyncSession = Depends(get_db),
):
    deleted = await ticket_service.delete_ticket(ticket_id, db)

    if not deleted:
        raise TicketNotFoundError(str(ticket_id))

    return {"message": "Ticket deleted successfully"}
```

Summarize service by Amazon Bedrock

```
@router.post("/summarize", response_model=SummarizeResponse)
def summarize_ticket(payload: SummarizeRequest) -> dict[str, str]:
    service = BedrockService()

    try:
        return service.summarize_ticket(payload.ticket_description)
    except BedrockServiceError as exc:
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail="The AI service is temporarily unavailable",
        ) from exc
```

Create Ticket Service

```
async def create_ticket(
    data: CreateTicketRequest,
    db: AsyncSession,
):
    ticket = Ticket(
        title=data.title,
        priority=data.priority,
        status="open",
        assignee=data.assignee
    )

    db.add(ticket)
    await db.commit()
    await db.refresh(ticket)

    return ticket
```

Get All Ticket Service

```
async def get_all(db: AsyncSession):
    result = await db.execute(select(Ticket))
    return result.scalars().all()
```

Get Ticket By Id service

```
async def get_ticket(
    ticket_id: UUID,
    db: AsyncSession,
):
    result = await db.execute(
        select(Ticket).where(Ticket.id == ticket_id)
    )

    return result.scalar_one_or_none()
```

Update Ticket Service

```
async def update_ticket(
    ticket_id: UUID,
    data: UpdateTicketRequest,
    db: AsyncSession,
):
    ticket = await get_ticket(ticket_id, db)

    if ticket is None:
        return None

    updates = data.model_dump(exclude_unset=True)

    for key, value in updates.items():
        setattr(ticket, key, value)

    await db.commit()
    await db.refresh(ticket)

    return ticket
```

Update Ticket Service

```
async def delete_ticket(
    ticket_id: UUID,
    db: AsyncSession,
):
    ticket = await get_ticket(ticket_id, db)

    if ticket is None:
        return False

    await db.delete(ticket)
    await db.commit()

    return True
```

References

The following official documentation and third-party resources were referenced during the design and implementation of the AI Service Desk Backend application:

- FastAPI Documentation – <https://fastapi.tiangolo.com/>
- SQLAlchemy Documentation – <https://docs.sqlalchemy.org/>
- Alembic Documentation – <https://alembic.sqlalchemy.org/>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Pydantic Documentation – <https://docs.pydantic.dev/>
- Pytest Documentation – <https://docs.pytest.org/>
- Docker Documentation – <https://docs.docker.com/>
- Uvicorn Documentation – <https://www.uvicorn.org/>
- AWS Documentation – <https://docs.aws.amazon.com/>
- Amazon Bedrock Documentation – <https://docs.aws.amazon.com/bedrock/>
- Boto3 Documentation – <https://boto3.amazonaws.com/>

These references provide authoritative guidance on the frameworks, libraries, cloud services, and development tools used throughout the implementation of the AI Service Desk Backend.